

title: “Coleta multi-regional: contratos regionais, sessões e isolamento por OSS - Plan” type: fix date: 2026-08-13 status: em_execucao

Coleta multi-regional: contratos regionais, sessões e isolamento por OSS - Plan

Objetivo

Fazer o evento **Teste Curitiba** (10.220.30.9 , regional OUTRAS) coletar KPIs e VIPs sem regredir **TesteSantoAmaro** (10.220.50.9 , regional SP) e sem permitir que uma coleta iniciada para um evento grave dados ou checkpoints no evento seguinte.

Este plano foi reestruturado após a análise dos arquivos:

- har-oss-outros/har-monitoring-oss-tsl.har ;
- har-oss-outros/har-vips-oss-tsl.har ;
- data/logs/smart_events.log ;
- data/smart_events.db e bancos específicos dos dois eventos.

As antigas Fases 1 e 2 permanecem válidas e já foram executadas. A antiga Fase 3, centrada em criar novos resolvedores de nomes, foi retirada do caminho crítico: os nomes reais da task PM 2225 já casam exatamente com o inventário do evento.

Quando o sistema deve voltar a funcionar

Marco	Resultado esperado
Estado atual, depois das Fases 1 e 2	Alarmes funcionam; KPI e VIP de Curitiba ainda não
Fim da Fase 3	Troca SP ↔ Curitiba isolada, sem workers e checkpoints cruzados; ainda não garante dados de Curitiba
Fim da Fase 4	KPIs de Curitiba devem aparecer e continuar funcionando em SP — CONCLUÍDA em 14/08
Fim da Fase 5	KPIs + VIPs + alarmes devem funcionar em Curitiba e SP — primeiro marco funcional completo
Fim da Fase 6	Solução endurecida, dados contaminados tratados e rollout pronto para produção

Portanto, a resposta objetiva é: os KPIs são esperados a partir do fim da Fase 4; o sistema completo, incluindo VIP, é esperado a partir do fim da Fase 5. A Fase 6 não deve ser necessária para fazer os dados aparecerem; ela fecha riscos operacionais e corrige o histórico contaminado.

Evidências consolidadas

Estado dos bancos em 13/08/2026

Item	Teste Curitiba	Teste Santo Amaro
sites cadastrados	431	5
células cadastradas	6.242	33
kpi_measurements	0	128.735
vip_measurements no banco global	0	50.315
alarmes no banco do evento	2.503	3.297
objetos PM configurados na task	116	29 objetos ativos no último checkpoint

Monitoring / KPI — o mapeamento da task 2225 já funciona

O HAR de Monitoring mostra que a task 2225:

- existe e abre com HTTP 200;
- chama-se `TESTE FERRAMENTA CURITIBA (2225)` ;
- é `Measurement of Cell Performance` , período de 5 minutos;
- contém 116 objetos e 24 contadores LTE usados pelas fórmulas do aplicativo;
- executou dois `POST /rest/oss/access/pm/v1/monitor/task/result` com HTTP 200 no navegador;
- possui sete sites: `CTFA01` , `CTFD01` , `CTFD60` , `CTFE01` , `CTFG99` , `CTFR99` e `CTFZ01` .

As 116 ocorrências de `Cell Name` foram comparadas com as células do evento:

- correspondência exata: **116/116**;
- execução do resolvedor atual com os nomes reais: **116 mapeadas, 0 descartadas**;
- não existe necessidade comprovada de remover prefixo, usar substring ampla ou cadastrar `obj_no` para essa task.

Uma NE estava desconectada: `SR-CTFD60` , com 12 objetos. Por isso o segundo ciclo do navegador consultou 104 objetos ativos. Essa indisponibilidade explica cobertura parcial, mas não explica zero KPIs — as outras 104 células deveriam produzir dados.

O bloqueio observado no aplicativo acontece antes do parser: a sessão de Monitoring recebe 401, o processo interpreta um `roarand` diferente no arquivo como renovação suficiente, recarrega a sessão e

recebe 401 novamente. O navegador, no mesmo OSS, recebe HTTP 200.

VIP — o OSS de Curitiba usa outro contrato FARS

A task 14837 não está vazia nem parada:

- `pre-check` : `checkState=true` ;
- `estado`: `Running` ;
- aproximadamente 388.997 mensagens no momento da captura;
- janela observada: `2026-08-13 09:37:02` até aproximadamente `18:15` ;
- a interface conseguiu filtrar `RRC_MEAS_RPRT` , ordenar e abrir mensagens decodificadas.

SP usa o contrato síncrono já implementado:

1. `GET query/result` ;
2. `GET query/fetch-field-values` ;
3. `POST query/filter-by-cols` ;
4. `POST query/sort` ;
5. `GET query/result-paging` .

Curitiba usa o contrato assíncrono:

1. `GET query/result` ;
2. `GET query/fetch-field` — **síncrono**, já devolve `startTime` / `endTime` ;
3. `POST query/filter-by-cols-start` ;
4. polling em `POST query/filter-by-cols-result` ;
5. `POST query/sort` ;
6. `GET query/result-paging` .

O navegador também faz polling em `GET query/fetch-field-values-result` entre 2 e 3, mas só para preencher os combos de filtro da tela (`filterMap`). O coletor filtra por um valor fixo e não lê esse campo — ver a Fase 5.

O coletor chama `fetch-field-values` , que não é o contrato dessa regional, e recebe HTTP 500 em todos os ciclos. Ele nem chega à filtragem, paginação ou decodificação. Este é o bloqueio direto do VIP de Curitiba.

Troca de projeto — coleta antiga grava no evento novo

Há evidência temporal direta:

- `18:29:19` : o scheduler registra a coleta anterior como encerrada e ativa Curitiba;
- `18:29:32` : a coleta VIP antiga de SP ainda termina com cinco medições da task 2073;
- no mesmo segundo, o checkpoint da task SP é gravado no banco de Curitiba com `oss='OUTRAS'` .

A causa é a combinação de:

- `stop()` faz `join(timeout=5)`, embora uma requisição possa durar 30–120 segundos;
- `start()` limpa o mesmo `_stop_event` logo depois;
- `_apply_result()` usa `_event_config` mutável, que já aponta para o evento novo quando o resultado antigo termina.

Também foram encontrados checkpoints PM 2225 sob `SP` e `OUTRAS` no mesmo banco, checkpoints VIP 2072/2073 de SP sob `OUTRAS`, dois eventos simultaneamente `ACTIVE` e várias ocorrências de `database is locked` durante sobreposição de workers.

Limitação das capturas HAR

O DevTools preservou metadados, payloads e rotas, mas não preservou todos os corpos comprimidos:

- Monitoring: 22 respostas sem `content.text`, incluindo as duas respostas de `monitor/task/result`;
- VIP: 19 respostas sem `content.text`, incluindo as respostas finais dos dois pollings assíncronos.

A mensagem `Failed to load response data / No data found for resource with given identifier` indica que o DevTools já não possui o corpo associado àquela entrada. Salvar novamente o mesmo log como HAR não recupera esse conteúdo. Por isso o novo plano não depende de obter esses corpos manualmente pelo navegador.

Regras de segurança e regressão

1. Cada fase deve ser um commit isolado.
 2. Antes de cada mudança, registrar o baseline atual da suíte; o gate é **nenhuma falha nova**, em vez de depender de uma contagem fixa que pode mudar com os testes adicionados.
 3. Validar sempre SP antes de Curitiba em cada rollout.
 4. Nunca apagar checkpoints ou bancos antes de criar backup e validar os alvos exatos.
 5. Adaptadores regionais devem ser detectados por contrato/capacidade, não por IP hardcoded.
 6. Logs e fixtures não podem conter `bspession`, `roarand`, cookies, IMSI ou credenciais.
 7. Um resultado deve carregar identidade imutável de evento, OSS e geração desde o início da coleta até a persistência. Estado global mutável não pode decidir onde gravá-lo.
-

Fase 1 — Observabilidade persistente — CONCLUÍDA

Entregas

- log rotativo em `data/logs/smart_events.log` ;
- captura crua opt-in e sanitizada em `data/diagnostics/` ;
- log individual para cada task VIP, inclusive vazia ou com falha;
- cobertura de Monitoring visível na interface;
- botão de captura de diagnóstico;
- testes de log, dump e interface.

Resultado obtido

A fase revelou o erro repetido de VIP em `fetch-field-values` , os 401 de Monitoring, a coleta antiga continuando após a troca e as ocorrências de bloqueio do SQLite.

Fase 2 — Cursor não avança sobre dado descartado — CONCLUÍDA

Entregas

- cursor por objeto somente quando o objeto produz pelo menos uma medição válida;
- cursor geral da task retido quando existe descarte;
- scheduler não confirma checkpoint de um ciclo totalmente descartado;
- persistência idempotente para permitir replay.

Limite da fase

Ela protege respostas que chegaram ao parser. Não pode corrigir:

- HTTP 401 antes de o Monitoring devolver JSON;
 - HTTP 500 do VIP antes de o filtro;
 - resultado de worker antigo aplicado ao evento novo.
-

Fase 3 — Isolamento determinístico ao trocar de evento

Objetivo

Eliminar a contaminação cruzada antes de continuar os testes de regional. Sem isso, qualquer validação de sessão ou API pode gravar no banco errado e produzir conclusões falsas.

Código

- `core/scheduler.py` :
 - introduzir um token/geração imutável por `start()` ;
 - cada worker captura `event_id` , regional, coletor e geração na criação;
 - antes de persistir ou atualizar status, descartar resultados de geração obsoleta;
 - `stop()` sinaliza a geração e não reutiliza o mesmo evento de parada para a próxima;
 - substituir a dependência de `_event_config` mutável dentro de `_apply_result` por contexto do próprio ciclo;
 - garantir no máximo um worker vivo por tipo e geração.
- `api/api.py::activate_event` :
 - encerrar o evento anterior antes de ativar o novo;
 - manter somente um evento `ACTIVE` ;
 - não iniciar B até a geração de A estar invalidada.
- `core/collector.py` e `core/session_renew.py` :
 - chavear `_needs_interactive` , backoff, falhas e cooldown por `(host, módulo)` ;
 - usar `browser_profile_<host>` em vez de um perfil compartilhado entre regionais.

Testes obrigatórios

- resultado de A terminado depois de ativar B não grava medição, alerta ou checkpoint em B;
- `stop()` + `start()` não reanima thread da geração anterior;
- ativar B encerra A no banco;
- backoff/CAPTCHA de SP não bloqueia Curitiba;
- perfis e session files permanecem separados por host.

Validação ao vivo

Executar SP → Curitiba → SP, verificando no log que nenhuma task da regional anterior termina aplicada após a ativação da nova. Conferir os checkpoints dos dois bancos.

Critério de saída

Nenhum estado cruzado em três trocas consecutivas. Esta fase torna os testes confiáveis, mas ainda não promete KPI ou VIP de Curitiba.

Commit sugerido

fix: isolate collection generations and sessions across OSS switches

Fase 4 — Sessão de Monitoring validada por host e retomada dos KPIs — CONCLUÍDA (14/08)

Resultado obtido

A sessão por host resolveu os 401, mas **não** fez os KPIs aparecerem: o bloqueio seguinte estava no parser. O OSS de Curitiba identifica a célula em `objRes[].obj.objectNo / objectName`, enquanto SP usa `objNo / objName` — e Curitiba ainda devolve `objName: null` no mesmo dicionário. Lendo só a primeira grafia, todo objeto caía em `int(None)` e era contado como inválido antes de `received`: HTTP 200, 116 objetos por janela, zero medição, zero log.

Isso é o cenário previsto no critério de saída abaixo ("se o JSON real revelar uma forma de `objName` diferente"). A correção foi `_obj_field`, que aceita as duas grafias, mais o log por ciclo e o dump automático que tornaram a causa visível. Replay do corpo real: 464 recebidos, 0 não mapeados, 5.124 linhas. Detalhes em [ERRORS.md](#) e [MEMORY.md](#) (14/08).

Objetivo

Fazer a PM task 2225 chegar ao parser com uma sessão realmente autenticada. O resolvedor existente já deve mapear os objetos.

Código

- `core/collector.py` :
 - não considerar mudança de `roarand` prova suficiente de renovação;
 - após recarregar ou renovar, executar um probe autenticado do módulo e só retornar sucesso após HTTP 200 com contrato JSON esperado;
 - se o probe falhar, continuar a renovação daquele `(host, módulo)` em vez de aceitar a sessão;
 - manter Monitoring e Trace independentes, embora compartilhem cookies quando válido;
 - distinguir nos logs `arquivo recarregado`, `probe aceito`, `probe recusado` e `Playwright usado`.
- `core/session_renew.py` :
 - gravar atomicamente o session file específico do host;
 - confirmar que o módulo solicitado capturou cookies e `roarand` válidos antes de retornar sucesso.
- preservar `_resolve_monitoring_cell` como está; não adicionar fallback amplo sem uma célula real que falhe no resolvedor exato.

Testes obrigatórios

- `roarand` mudou, mas probe retorna 401 → renovação ainda não é sucesso;
- probe HTTP 200/JSON válido → sessão aceita;
- renovação de Trace não marca Monitoring inválido como renovado;
- os 116 objetos reais extraídos da abertura da task mapeiam 116/116;
- 12 objetos indisponíveis não impedem as outras 104 células de produzir medições;
- resposta vazia legítima mantém a semântica de cursor da Fase 2.

Validação ao vivo

1. SP por dois ciclos: KPIs continuam entrando.
2. Curitiba por até dois ciclos PM:
 - HTTP 200 no log;
 - `recebidos > 0`, `mapeados > 0`;
 - linhas em `kpi_measurements`;
 - cursor volta a avançar apenas para objetos persistidos.
3. Confirmar que `SR-CTFD60` aparece como indisponibilidade parcial, não como falha total.

Critério de saída — PRIMEIRO MARCO FUNCIONAL

KPIs de Curitiba aparecem no mapa e no banco, sem regressão em SP. Se o JSON real revelar uma forma de `objName` diferente da resposta de abertura da task, abrir uma correção mínima apoiada na captura — não reativar automaticamente toda a antiga Fase 3.

Commit sugerido

```
fix: validate monitoring sessions per OSS before collecting PM data
```

Fase 5 — Adaptador FARS síncrono/assíncrono e retomada dos VIPs

Objetivo

Consumir a task 14837 no contrato assíncrono de Curitiba sem alterar o caminho síncrono que funciona em SP.

Estado de partida (14/08, depois da Fase 4)

Todo ciclo de VIP em Curitiba morre no mesmo ponto, a cada 60 s:

```
[vip/20260813_Testes] task 14837: recordCount=0 linhas_lidas=0 linhas_decodificadas=0  
motivo=HTTP 500 .../fars/v1/traceresult/query/fetch-field-values?taskId=14837&msgId=359008
```


O `msgId` incrementa a cada ciclo (359008, 360008, 361008...), ou seja: `pre-check` e `query/result` já funcionam em Curitiba. Só o passo seguinte não existe nessa regional.

O delta real são dois endpoints, não sete

Comparando `core/collector.py` (implementado, SP) com `har-oss-outros/har-vips-oss-tsl.har` (capturado, Curitiba — 83 entradas, task 14837):

Passo	Método SP (hoje)	Método Curitiba (HAR)	Muda?
pre-check	<code>GET tracersresult/pre-check</code> (fora de <code>query/</code>)	idêntico, <code>{"checkState":true,"checkResult":[]}</code>	não
abrir consulta	<code>GET query/result</code> com <code>msgId=1</code> → <code>data.msgId</code>	idêntico (alocou 325008)	não
janela da task	<code>GET query/fetch-field-values</code>	<code>GET query/fetch-field</code>	sim
filtrar	<code>POST query/filter-by-cols</code>	<code>POST query/filter-by-cols-start</code> + polling <code>POST query/filter-by-cols-result</code>	sim
ordenar	<code>POST query/sort</code> → NOVO <code>msgId</code>	idêntico (328008 → 329008), síncrono	não
paginar	<code>GET query/result-paging</code>	idêntico, síncrono	não

Não existe `sort-start` / `sort-result` nem `result-paging-start` no HAR: fora dos dois passos marcados, o contrato de Curitiba é o mesmo. `query/result/color` e `query/result/operate-columns` são da tela, o coletor não precisa deles.

Correção importante ao plano anterior: o polling da janela não é necessário

A versão anterior desta fase mandava usar `fetch-field` e fazer polling em `fetch-field-values-result` . Isso está errado e custaria trabalho à toa. `GET query/fetch-field` responde **na hora**, com exatamente o que o coletor precisa:

```
{
  "startTime": "2026-08-13 09:37:02",
  "endTime": "2026-08-13 18:15:04",
  "filterMap": {"Trace Type": null, "Message Direction": null, "Call ID": null,
               "Mode": null, "Cell ID": null, "Source": null, "Message Type": null},
  "signalList": []
}
```

`startTime` / `endTime` vêm no mesmo formato e nas mesmas chaves que o `fetch-field-values` de SP — `_open_trace_query` só usa esses dois campos. O que o navegador busca depois, com quatro polls em `fetch-field-values-result` (`{"process":10,"filterMap":{}}`), são os **valores dos combos de filtro** (`filterMap` preenchido). O aplicativo filtra por um valor fixo, `RRC_MEAS_RPRT` , e nunca leu `filterMap` .
Não implemente esse polling.

O corpo do filtro é byte a byte o que o app já monta

POST `query/filter-by-cols-start`, sem query string além de `nocache`:

```
{
  "colFilterDto": {
    "colFltExpSeq": [{ "fieldId": "Message Type", "value": "RRC_MEAS_RPRT", "operator": { "op": 0 } }],
    "signalList": [], "hasStartTime": false, "startTime": "2026-08-13 09:37:02",
    "hasEndTime": false, "endTime": "2026-08-13 18:15:04", "isReverse": false
  },
  "pageDto": {
    "sqlColumnName": "", "isAscend": "", "taskId": 14837, "msgId": 325008,
    "comparisonMsgId": -1, "startRow": 0, "pageSize": 1000,
    "templateName": [], "isSetBenchmarkTime": false, "benchmarkTimeRowNo": -1
  }
}
```

Isso é **idêntico** ao corpo que `_filter_meas_reports` já envia hoje ([`core/collector.py:1834`]). Só a URL muda. `pageDto.msgId` é o handle aberto em `query/result`; `startTime` / `endTime` vêm de `fetch-field` mesmo com `hasStartTime` / `hasEndTime` em `false` (ver [ERRORS.md](#), 12/08 — vazio dá 500).

O polling do filtro, e a única coisa que o HAR não tem

Poll: POST `query/filter-by-cols-result`, corpo `{"msgId": 325008}` — o `msgId` de entrada, não um novo. Envelopes capturados, em ordem, ~0,6 a 1,0 s de intervalo:

```
{"status":1,"errorMsg":null,"progress":10,"value":null}
{"status":1,"errorMsg":null,"progress":10,"value":null}
{"status":1,"errorMsg":null,"progress":65,"value":null}
{"status":1,"errorMsg":null,"progress":75,"value":null}
```

O quinto poll — o da conclusão — **está sem corpo no HAR** e não é recuperável (ver a seção sobre a limitação das capturas). É o único desconhecido desta fase. O que se sabe dele por inferência direta: a requisição seguinte do navegador foi `result-paging` com `msgId=328008`, então o envelope de conclusão carrega o novo handle, quase certamente em `value`. `status=1` é “em progresso”; o código terminal não foi observado.

Não trave a fase esperando essa captura. Implemente a leitura tolerante — ela cobre as formas plausíveis sem adivinhar nenhuma:

1. conclusão = `value` deixou de ser `null` ou `status` mudou de `1`;
2. erro = `errorMsg` não nulo (aborta o ciclo, sem cursor);
3. do envelope de conclusão, extrair o `msgId` procurando recursivamente a primeira chave `msgId` com valor inteiro em `value` → `data` → raiz; se `value` for um inteiro puro, ele é o `msgId`;
4. se nada disso render um `msgId`, **falhar o ciclo com log explícito e gravar o envelope inteiro em `data/diagnostics/`** — nunca seguir para o `sort` com o handle antigo, que devolveria o conjunto não filtrado.

O caso 4 é a rede de segurança: se a forma real for outra, o primeiro ciclo ao vivo entrega o envelope no diagnóstico e o ajuste vira uma linha. Isso substitui o antigo “Passo 5A” — o probe separado deixou de

ser pré-requisito porque o dump automático da Fase 4 já grava corpo sanitizado (`_dump_raw` , `data/diagnostics/`).

Parâmetros de polling, calibrados pela captura (filtro de 388.997 mensagens levou ~3,5 s): intervalo 0,5 s, prazo total 120 s, e checagem de cancelamento pela geração da Fase 3 a cada volta. Um `filter-by-cols-start` por ciclo — repetir re-executa o filtro contra dados vivos e mistura snapshots.

Armadilha confirmada: `null` onde SP manda número

A resposta de página vazia em Curitiba é:

```
{"recordCount":null,"data":{"lastSerialNo":null,"msgId":328008,"tableData":[],"serialNo":null}}
```

`recordCount` vem `null` , não `0` . `_allocated_msg_id` já absorve (`int(... or 0)`), mas dois pontos precisam de conferência antes de dar a fase por fechada:

- `_open_trace_query` ([`core/collector.py:1783`]) encerra o ciclo como vazio quando `recordCount == 0` no bootstrap. Se Curitiba mandar `null` ali, uma task com 388.997 mensagens vira “ciclo vazio” em silêncio. Hoje isso não acontece — o ciclo chega ao passo da janela — mas confirme no log do primeiro ciclo bom.
- `serialNo` / `lastSerialNo` nulos não podem virar cursor. `_build_vip_measurements` já trata (`int(item.get("serialNo") or -1)`), mas veja o item seguinte.

Verifique a grafia dos campos da linha antes de confiar no decoder

Os corpos de `result-paging` com `tableData` cheio não sobreviveram no HAR. O contrato por linha que `_build_vip_measurements` consome é:

Campo	Onde	Uso
<code>serialNo</code>	raiz da linha	cursor e dedup (<code>task_id</code> , <code>serial_no</code>)
<code>messageBody</code>	raiz da linha	<code>decode_meas_report</code> (RSRP/RSRQ)
<code>msgType</code> OU <code>payload[].GULTrcMsgType</code>	raiz / lista	confirmar <code>RRC_MEAS_RPRT</code>
<code>payload[].GLCellId</code>	lista {name,value}	célula servidora
<code>payload[].Time</code>	lista {name,value}	timestamp
<code>source</code>	raiz da linha	prefixo da célula servidora

Este é exatamente o formato da falha que custou a Fase 4: o Monitoring de Curitiba manda `objectNo` / `objectName` onde SP manda `objNo` / `objName` , e o parser rejeitou 100% dos objetos em silêncio. Antes de declarar a fase pronta, abra o primeiro `result-paging` não vazio gravado em `data/diagnostics/` e confira as seis chaves acima uma a uma. Se alguma divergir, resolva com um leitor que aceite as duas grafias (`_obj_field` em `core/collector.py` é o precedente), nunca trocando a grafia de SP.

Passo 5A — adaptador de contrato

Pontos de alteração, todos em `HttpCollector` :

- `_open_trace_query` ([`core/collector.py:1738`]): o passo da janela vira despacho pelo adaptador.
- `_filter_meas_reports` ([`core/collector.py:1825`]): mesmo corpo, URL e conclusão pelo adaptador.
- `_sort_trace_by_time`, `_fetch_trace_page`, `_build_vip_measurements`, `collect_vips` : **não tocar** — o contrato desses passos é igual nas duas regionais.

Seleção do adaptador **por capacidade, nunca por IP** (regra 5 deste plano):

1. tentar `GET query/fetch-field-values` ; HTTP 200 com `startTime` / `endTime` → host é síncrono;
2. HTTP 500/404/405 → tentar `GET query/fetch-field` ; HTTP 200 com `startTime` / `endTime` → host é assíncrono;
3. gravar a decisão em cache por (`host`, `versão da sessão`) , como `_needs_interactive` faz por (`host`, `módulo`) desde a Fase 3, e logar `[vip] contrato=sincrono|assincrono host=...` na escolha e a cada troca;
4. o passo do filtro usa a decisão em cache; se mesmo assim a URL responder 404, cair para a outra forma uma vez e regravar o cache.

A ordem importa: tentar o síncrono primeiro garante que SP não muda de caminho.

Testes obrigatórios

- SP: a sequência atual continua idêntica, chamando `fetch-field-values` e `filter-by-cols` ;
- detecção: 500 em `fetch-field-values` leva a `fetch-field` e marca o host como assíncrono;
- detecção fica em cache — o segundo ciclo do mesmo host não repete a tentativa síncrona;
- polling percorre `progress` 10 → 65 → 75 → conclusão e extrai o `msgId` novo;
- `value` inteiro puro e `value` objeto com `msgId` são ambos aceitos (leitura tolerante);
- envelope de conclusão sem `msgId` reconhecível → ciclo falha, grava diagnóstico e **não** segue para o `sort` ;
- `errorMsg` preenchido, timeout do prazo total e cancelamento por troca de geração: nenhum grava cursor `row` nem `serial` ;
- um único `filter-by-cols-start` por ciclo;
- `recordCount: null` e `serialNo: null` não viram cursor nem contagem falsa;
- fixtures dos dois contratos, sanitizadas, entrando pelo mesmo ponto que a resposta real entra (fixture que já chega desembulhada não testa o desembulho — ver [ERRORS.md](#), 14/08).

Validação ao vivo

1. SP: tasks 2072 e 2073 continuam coletando, com a mesma sequência no log.
2. Curitiba, task 14837:
 - `[vip] contrato=assincrono` no log;
 - passa da janela sem HTTP 500;
 - `recordCount` filtrado > 0 ;
 - `linhas_lidas > 0` e `linhas_decodificadas > 0`;
 - medições no painel VIP e em `vip_measurements`.
3. Trocar de evento durante um polling e confirmar que o ciclo cancelado não persiste nada.

Critério de saída — SISTEMA FUNCIONAL COMPLETO

Alarmes, KPIs e VIPs funcionam em SP e Curitiba. Este é o primeiro ponto em que a coleta multi-regional completa deve estar operacional.

Commit sugerido

`fix: support synchronous and asynchronous FARS query contracts`

Fase 6 — Endurecimento, saneamento e rollout

Objetivo

Tratar o histórico contaminado e fechar os riscos restantes depois que a coleta já estiver funcionando.

Ações

- criar backups datados dos bancos antes de qualquer limpeza;
- produzir relatório exato dos checkpoints incompatíveis, sem apagá-los automaticamente;
- remover, após validação explícita:
 - tasks VIP 2072/2073 sob `OUTRAS` no banco de Curitiba;
 - registros PM 2225 sob `SP` no banco de Curitiba;
 - outros checkpoints cuja task não pertence ao evento/OSS;
- manter medições históricas válidas e usar as chaves únicas para replay seguro;
- resolver os `database is locked` restantes:
 - transações curtas;
 - rollback garantido;
 - WAL/busy timeout se confirmado necessário pelos testes;
- fazer `get_event_vips` falhar fechado quando cliente/regional não estiver resolvido;
- remover fallback silencioso de `resolve_base_url` para SP;
- exibir na interface regional/host ativo, contrato FARS selecionado e causa por task;
- atualizar `MEMORY.md` e `ERRORS.md` com os dois contratos regionais.

Validação final

1. Rodar SP por três ciclos completos.
2. Rodar Curitiba por três ciclos completos.
3. Repetir SP → Curitiba → SP.
4. Confirmar:
 - somente um evento `ACTIVE` ;
 - nenhum worker antigo vivo;
 - nenhum checkpoint cruzado novo;
 - KPIs e VIPs avançando sem lacunas ou duplicação;
 - alarmes continuam chegando;
 - nenhum novo `database is locked` .

Commit sugerido

fix: harden multi-regional persistence and clean invalid checkpoints

Como recapturar no DevTools, se ainda for útil

Isso deixa de ser bloqueador, mas uma nova captura pode ajudar na contraprova:

1. abrir o DevTools **antes** de abrir a task;
2. em Network, ativar gravação, `Preserve log` e `Disable cache` ;
3. limpar a lista;
4. filtrar por `fetch-field-values-result` , executar a ação e abrir a entrada mais nova assim que concluir;
5. repetir para `filter-by-cols-result` ;
6. salvar imediatamente com **Save all as HAR with content**.

Se a aba Response já mostrar `No data found for resource with given identifier` , aquele corpo já foi perdido e não há configuração que o recupere retroativamente. Não é necessário insistir: o probe da Fase 5A é a fonte preferencial e mais confiável.

Decisões que este plano substitui

- Não tratar a antiga Fase 3 de mapeamento como correção principal da PM 2225: os 116 nomes reais já casam exatamente.
- Não esperar que a Fase 2 faça os dados aparecerem: ela protege cursores depois do parser, enquanto os bloqueios atuais acontecem antes dele.
- Não deixar isolamento de threads e sessões para o final: a contaminação já aconteceu e invalida testes intermediários.
- Não depender de HAR com corpos grandes para implementar o FARS assíncrono: capturar diretamente no cliente HTTP do aplicativo.
- Não fazer polling em `fetch-field-values-result` : `fetch-field` já devolve a janela na hora, e aquele polling só preenche combos da tela.
- Não tratar o probe do antigo Passo 5A como pré-requisito: o dump automático da Fase 4 grava o envelope do primeiro ciclo ao vivo, e a leitura tolerante do `filter-by-cols-result` permite implementar antes de conhecer o corpo de conclusão.